

UNIT – 4  
ASSESSMENT – 3  
REPORT

|                                |
|--------------------------------|
| NIHARIKA PREEJESH<br>192524506 |
|--------------------------------|

**Title 1:**

**Decision Tree Classification Using Iris Dataset**

**Aim**

To implement and evaluate a Decision Tree classifier using the Iris dataset and analyze its classification performance.

**Objective**

- To load and preprocess the Iris dataset.
- To divide the dataset into training and testing sets.
- To construct a Decision Tree using the Gini Index.
- To identify the root node.
- To evaluate classification performance using accuracy, confusion matrix and classification report.
- To identify misclassified instances.

**Algorithm**

1. Load the Iris dataset.
2. Store the features and target values.
3. Display the first five rows.
4. Display the data types.
5. Check for missing values.
6. Split the dataset into 80% training and 20% testing data.

7. Create a Decision Tree classifier using the Gini criterion.
8. Train the classifier.
9. Print the tree structure.
10. Identify the root node.
11. Predict the classes of the testing data.
12. Calculate accuracy.
13. Generate the confusion matrix.
14. Generate the classification report.
15. Identify misclassified instances.
16. Display feature importance.

## **Performance Metrics**

| <b>Metric</b>    | <b>Purpose</b>                          |
|------------------|-----------------------------------------|
| Accuracy         | Overall correct predictions             |
| Confusion Matrix | Correct and incorrect class predictions |
| Precision        | Correct positive predictions            |
| Recall           | Correctly identified actual positives   |
| F1-Score         | Balance between precision and recall    |

## **Result**

The Decision Tree successfully classified the Iris flowers into their respective species. The model achieved high classification accuracy and the confusion matrix showed very few or no misclassified instances.

## **Conclusion**

The Decision Tree classifier is effective for the Iris classification problem. The Gini Index helps select suitable splitting features, while the tree structure makes the classification process easy to interpret.

## **Title 2:**

### **Reinforcement Learning Using Q-Learning in a 4×4 Grid World**

#### **Aim**

To implement the Q-Learning reinforcement learning algorithm in a 4×4 Grid World and observe how the agent learns an optimal policy through repeated interaction with the environment.

#### **Objective**

- To create a 4×4 Grid World environment.
- To define states and actions.
- To assign rewards for different actions.
- To initialize and update a Q-table.
- To train the agent for at least 100 episodes.
- To observe the evolution of Q-values.
- To extract the final learned policy.
- To analyze the convergence using cumulative rewards.

---

#### **Environment**

| <b>Element</b> | <b>Value</b> |
|----------------|--------------|
| Grid           | $4 \times 4$ |

| Element     | Value                 |
|-------------|-----------------------|
| Start       | State 0               |
| Goal        | State 15              |
| Obstacle    | State 6               |
| Actions     | Up, Down, Left, Right |
| Goal Reward | +10                   |
| Normal Step | -1                    |
| Obstacle    | -5                    |

---

## Hyperparameters

| Parameter  | Value | Purpose                  |
|------------|-------|--------------------------|
| $\alpha$   | 0.1   | Learning rate            |
| $\gamma$   | 0.9   | Future reward importance |
| $\epsilon$ | 0.2   | Exploration probability  |
| Episodes   | 100   | Training iterations      |

---

## Algorithm

1. Initialize the Q-table with zeros.
2. Set the learning rate  $\alpha$ .
3. Set the discount factor  $\gamma$ .
4. Set the exploration rate  $\epsilon$ .
5. Start the agent at the initial state.

6. Select an action using the  $\epsilon$ -greedy strategy.
7. Perform the selected action.
8. Observe the reward and next state.
9. Calculate the maximum future Q-value.
10.     Update the Q-value using the Q-Learning equation.
11.     Continue until the goal is reached or the maximum number of steps is completed.
12.     Repeat the process for 100 episodes.
13.     Record Q-values at Episodes 1, 50 and 100.
14.     Select the highest-Q action for each state.
15.     Display the final learned policy.
16.     Plot cumulative reward to analyze convergence.

## **Q-Learning Update**

Where:

- $s$  = current state
- $a$  = current action
- $R$  = reward
- $s'$  = next state
- $\alpha$  = learning rate
- $\gamma$  = discount factor

**Result:**

The Q-Learning agent successfully learned action values by interacting with the Grid World environment. The Q-table was updated over 100 episodes and the learned policy was extracted by selecting the action with the highest Q-value for each state.

The cumulative reward generally improves as the agent gains experience, indicating that the agent is learning a better path toward the goal.